University and hospital administration

Technical report
Student: Nuriya Zhumanalieva
Assignment: Independent Work 1
Project: OOP-nemo
Date: 5 November 2026

Project purpose

This work extends the Person, Student, Teacher and Doctor system from Assignment 5 with the Java Collection Framework. It preserves the original domain relationships and payment formulas while adding indexed registries, sorting, request processing and consistent CRUD operations.

The result is a runnable Java console application with a deterministic demonstration, an interactive menu and a regression suite. All 184 assertions in 10 test groups passed. The original nine Java listings were recovered separately, compiled and executed for comparison.

Continuity with the previous project

Person remains abstract. Student, Teacher and Doctor retain their original fields and activities. Student creates and owns exam results; hospitals share independently created doctors. PayrollService still calculates payments through the Payable contract. Person now implements that common contract for the hierarchy.

The original Person[] array is replaced by HashMap and role-indexed sets. Immutable business IDs were added to entities. Payment methods use long to avoid int arithmetic overflow, and validation failures throw exceptions handled by the console. The original source is retained under reference/assignment5/src.

Scope

This is an in-memory, single-user coursework application. Requests represent simulated payouts, not bank transfers. Data is reset on restart. Persistence, login infrastructure and concurrent access are outside the assignment scope.

Repository: github.com/nelliewxs/OOP-nemo

Domain model and relationships

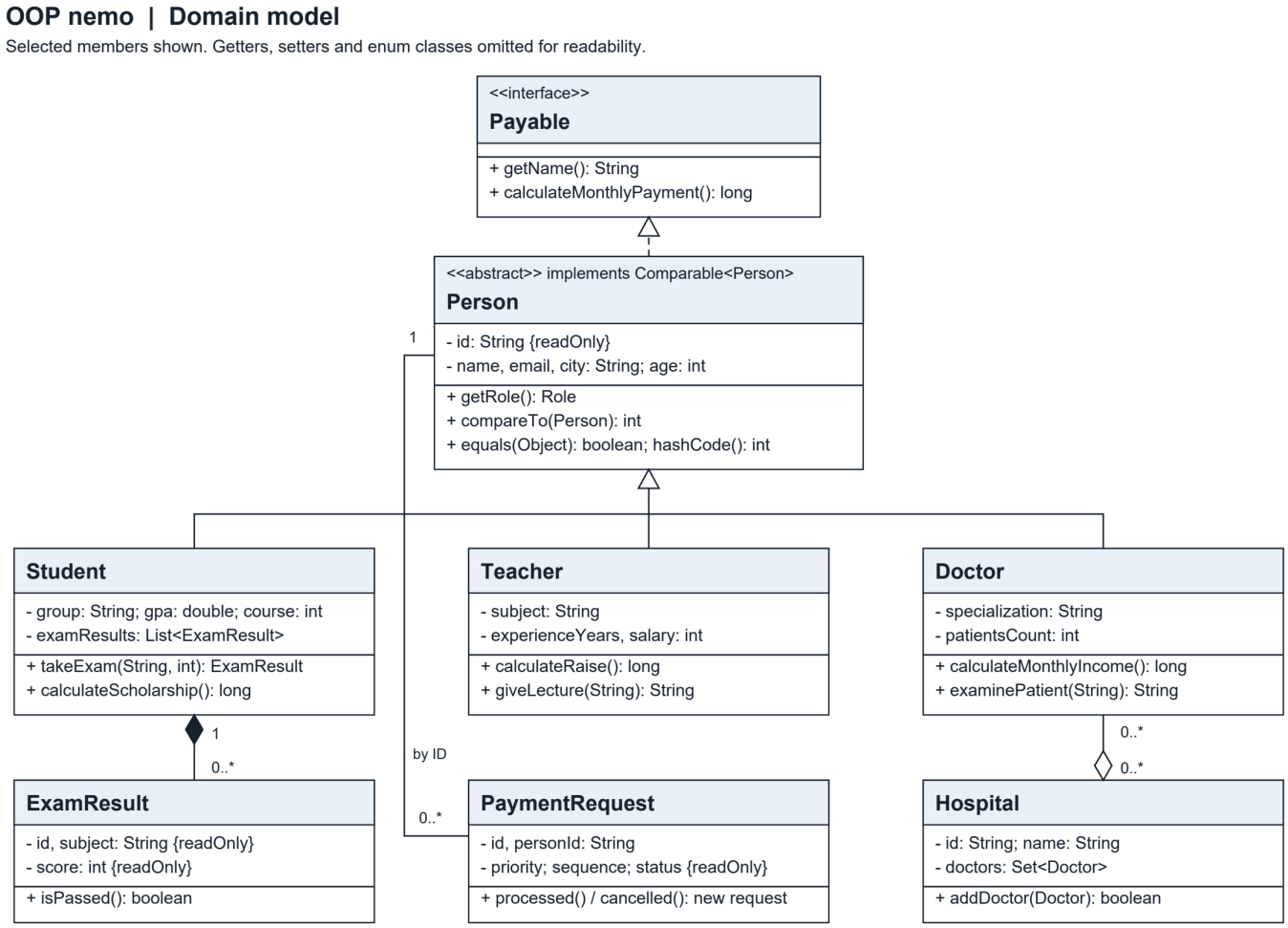


Figure 1. Updated UML domain model. The filled diamond denotes Student-owned ExamResult composition. The hollow diamond denotes Hospital-Doctor aggregation; both ends permit multiple memberships. Requests identify their applicant by immutable personId. Role is STUDENT, TEACHER or DOCTOR; priorities are HIGH, MEDIUM and LOW; statuses are PENDING, PROCESSED and CANCELLED.

Service layer and collection ownership

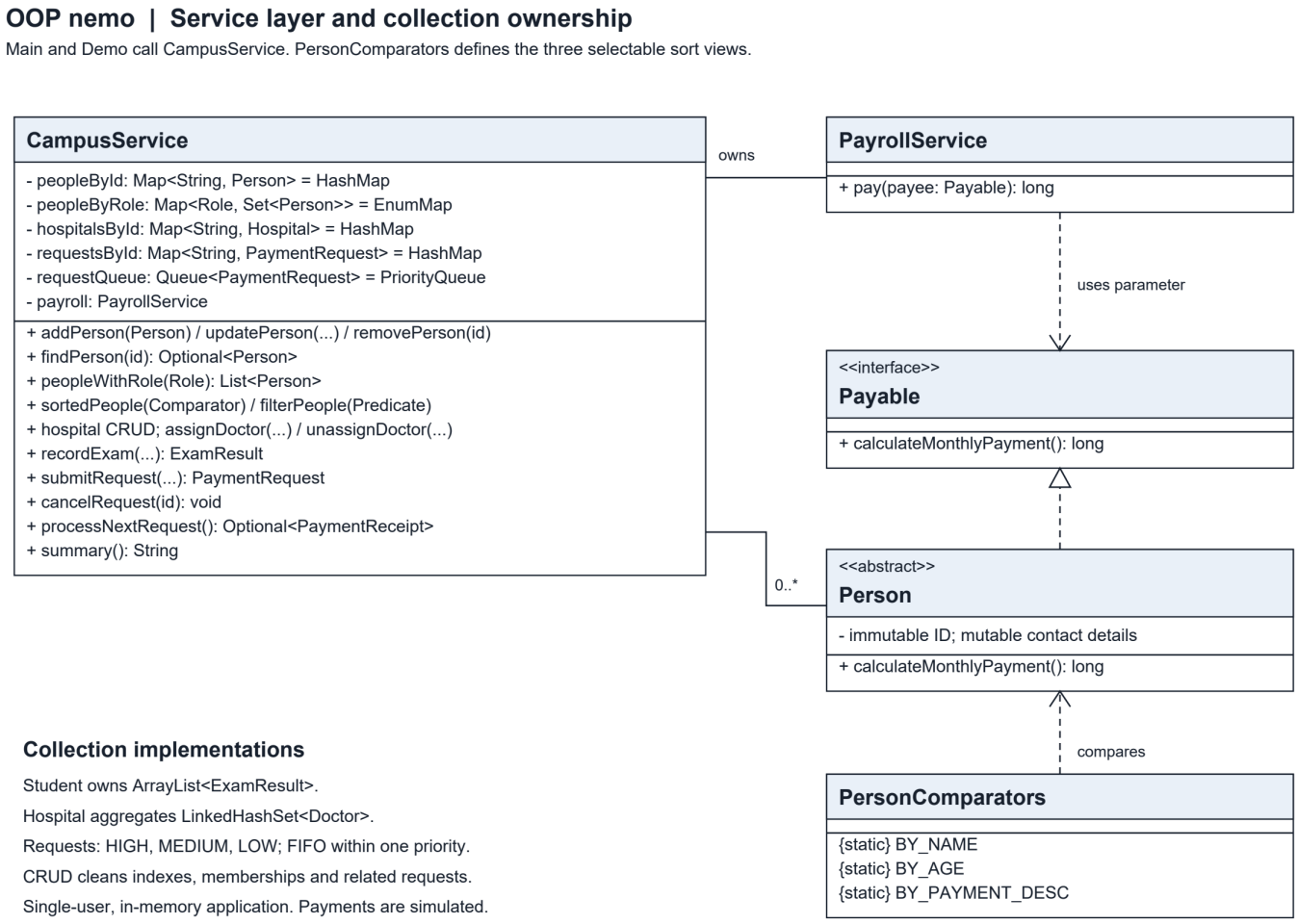


Figure 2. CampusService owns the registries and queue, coordinates CRUD and calls PayrollService. Main provides the applicant and payroll-operator actions. PersonComparators supplies three temporary sorted views. Collection types and selected operations match the implementation; routine accessors and validation utilities are omitted.

Collection choices and contracts

Collection	Purpose and justification
HashMap<String, Person>	Primary registry; expected O(1) key lookup. No duplicate general-purpose list is maintained.
EnumMap<Role, Set<Person>>	Categorization index. LinkedHashSet role members enforce uniqueness by immutable person ID.
ArrayList<ExamResult>	Ordered student-owned exam history; amortized O(1) append. Each retake gets a new result ID.
LinkedHashSet<Doctor>	Hospital membership without duplicates. A doctor can be shared across hospitals.
HashMap for hospitals and requests	Direct lookup of hospitals and current-session request history by business key.
PriorityQueue<PaymentRequest>	Priority processing with FIFO submission sequence for equal priorities. Sort keys are immutable.

Equality and ordering

Person, Hospital, ExamResult and PaymentRequest implement equals and hashCode using their immutable business IDs. All Person subclasses share one ID namespace. Changes to names, ages or payment attributes do not change identity or hash buckets.

Person implements Comparable by lexicographic ID, consistent with equality. Three custom comparators provide name ascending, age ascending and monthly payment descending, with ID tie breakers. They sort temporary lists; mutable display attributes are not stored as TreeSet ordering keys.

Collection getters return unmodifiable snapshots. Person objects remain shared canonical references, so changing a person's contact data is visible through the role index and hospital membership.

CRUD and role workflows

Consistent CRUD

Creation validates an entity and rejects an existing ID before changing registries. Contact updates validate every field before assigning any value, preventing partial changes. A person's role and ID do not change. The same object is referenced by the primary index, role set and hospital membership.

Deleting a person removes the primary entry, role membership, pending queue entries and all of that person's request history. Deleting a doctor also removes that doctor from every hospital. Removing a hospital leaves its independently registered doctors intact. Student-owned exam results leave the managed graph with their owner.

Applicant and operator workflow

A registered student, teacher or doctor submits a uniquely identified payment request. The service permits only one pending request per person. The payroll operator processes the next request: HIGH before MEDIUM before LOW, and earlier submissions first within one priority. The amount is calculated from the person's current details, then poll removes the queued request and the request index receives an immutable PROCESSED replacement.

Cancellation similarly replaces the request with a CANCELLED version and removes it from the queue. A processed or cancelled request cannot be cancelled again. An empty queue returns Optional.empty. If payment calculation fails, the request remains queued because calculation precedes removal.

Complexity and error handling

HashMap key lookup has expected O(1) cost under suitable hash distribution. PriorityQueue offer and poll cost O(log q); peek is O(1). Complete submission also scans existing requests for a pending duplicate, so it is O(r). Filtering and cascade deletion require scans; sorting costs O(n log n). Queue display sorts a copy because iterator order is unspecified.

Validation covers blank fields, email structure, finite GPA, age/course ranges, negative amounts, duplicate keys, missing records and wrong role operations. The menu handles invalid numeric input and end-of-input. Consistent CRUD is not a claim of thread safety.

Execution evidence for the domain model

Domain model, identity and sorting

Actual Java execution captured on 5 November 2026. Command: `java -cp build/classes oopnemo.app.Main --demo`

```
OOP NEMO | SIW1 | University and hospital administration

=== 1. Original classes, relationships and payment rules ===
D001 | DOCTOR  | Elena Petrova     | age 38 | elena@example.com | Karakol | payment 2900 USD
D002 | DOCTOR  | Azamat Bekov      | age 32 | azamat@example.com | Osh | payment 2100 USD
S001 | STUDENT | Aizada Omurova    | age 19 | aizada@example.com | Osh | payment 120 USD
S002 | STUDENT | Timur Asanov      | age 21 | timur@example.com | Bishkek | payment 65 USD
T001 | TEACHER | Nurlan Sadykov    | age 45 | nurlan@example.com | Bishkek | payment 1035 USD
Student-owned results: [Mathematics: 87 PASS, Physics: 41 FAIL]
Dr. Elena Petrova examined Demo patient. Total patients: 121
H001 | City Hospital No. 1 | doctors 1 -> [D001]
H002 | Private Clinic | doctors 2 -> [D001, D002]

=== 2. Business identity, primary lookup and role index ===
Same ID equals: true
HashSet size for two equal IDs: 1
Lookup D001: Elena Petrova
Doctors from role index: [D001, D002]
City filter Osh: [D002, S001]

=== 3. Natural ordering and three custom comparators ===
Natural ID order: [D001, D002, S001, S002, T001]
Name ascending: [S001, D002, D001, T001, S002]
Age ascending: [S001, S002, D002, D001, T001]
Payment descending: [D001, D002, T001, S001, S002]
```

Figure 3. Browser screenshot of captured Java --demo output. This is actual execution text, rendered in the evidence viewer. It demonstrates the original hierarchy, composition and aggregation, ID-based equality, primary and role lookup, filtering, natural order and all three custom sort views. Raw output is retained as docs/evidence/demo-output.txt.

Execution evidence for queue and CRUD

Priority workflow, CRUD and analytics

Actual Java execution captured on 5 November 2026. Command: `java -cp build/classes oopnemo.app.Main --demo`

```
=== 4. Applicant and payroll operator priority workflow ===
Submission order: LOW, HIGH, MEDIUM, HIGH
Processing order: priority first; FIFO within equal priority
R-HIGH-1 | person D001 | HIGH | PENDING
R-HIGH-2 | person D002 | HIGH | PENDING
R-MEDIUM | person T001 | MEDIUM | PENDING
R-LOW | person S001 | LOW | PENDING
Processed: R-HIGH-1 | Elena Petrova (D001) | 2920 USD
Processed: R-HIGH-2 | Azamat Bekov (D002) | 2100 USD
Processed: R-MEDIUM | Nurlan Sadykov (T001) | 1035 USD
Processed: R-LOW | Aizada Omurova (S001) | 120 USD
Empty queue returns Optional.empty: true

=== 5. CRUD consistency and edge cases ===
Updated name in hospital: Elena Petrova Updated
Deleted D001: primary lookup empty = true
Role index after deletion: [D002]
H001 memberships after deletion: 0
Pending requests after deletion: 0
Deleted person's request removed: true
Duplicate rejected: Person ID already exists: S001
Invalid GPA rejected: GPA must be finite and between 0 and 4.
Missing ID handled: Person not found: MISSING

=== 6. Aggregated report after CRUD ===
People: 4 | Students: 2 | Teachers: 1 | Doctors: 1
Hospitals: 2 | Pending requests: 0 | Processed requests: 3
Monthly payments: 3320 USD | Average student GPA: 3.50
Demo finished successfully. Payments are simulated; data is in memory.
```

Figure 4. Captured Java execution shows two HIGH requests processed in submission order, followed by MEDIUM and LOW. Updating the doctor is reflected in hospital membership; deletion then cleans the role index, memberships and requests. The final totals reflect the four remaining people. Three processed requests remain because the deleted doctor's history was removed.

Verification and reproducible execution

Regression suite

Actual Java execution captured on 5 November 2026. Command: `scripts/test.ps1`

```
Build passed (Java 17 target).
PASS Business identity and collection contracts
PASS Empty collections and missing keys
PASS CRUD and atomic validation
PASS Composition, aggregation and many-to-many
PASS Priority order, FIFO ties and cancellation
PASS Cascade removal across every managed reference
PASS Sorting, filters, analytics and snapshots
PASS Validation and payment arithmetic
PASS Repeated CRUD consistency with 100 people
PASS Interactive console with valid, invalid and ended input
PASS: 10 test groups; 184 assertions.
```

Figure 5. All 184 assertions in 10 groups passed. Tests cover equality, atomic updates, relationships, queue order and cancellation, cascade cleanup, analytics, validation, 100-person CRUD and interactive console subprocesses.

How to run

Install JDK 17 or newer. On Windows, run **scripts/run.ps1** for the menu, **scripts/run.ps1 -Demo** for the complete demonstration, or **scripts/test.ps1** for the regression suite. On Linux/macOS, use sh scripts/run.sh --demo and sh scripts/test.sh. The Java application needs no external libraries.

Local verification used JDK 23 with javac --release 17, UTF-8 and -Xlint:all -Werror. The generated bytecode targets Java 17. Windows scripts and the actual menu were exercised locally. GitHub Actions is configured to run the same suite on Java 17 and 23, on Windows and Linux; its run history records remote results.

Sources and submission materials

Course materials: SIW1.pdf and Assignment 5 ZNZ.pdf. Technical references: Oracle Java SE 17 API documentation for HashMap, PriorityQueue, Comparable and Comparator. Direct links and a detailed requirement matrix are in docs/requirements.md.

Submit the repository link, this report and docs/uml/class-diagram.pdf. The editable UML source and screenshots are included. docs/defense-ru.md provides oral-defense questions and a demonstration sequence. Oral understanding must be demonstrated by the student.